

Exercise

Kinematics and control of a differential drive vehicle

1. Closed-loop Control

In order to obtain a precise and smooth control of a differential-drive robot onto a reference pose, the linear state feedback control law described in the book *Introduction to Autonomous Mobile Robots* that accompanies this task can be implemented (see Chapter 3.6, p.91ff). The control law is summarized below. Variables correspond to the ones introduced in Figure 1.

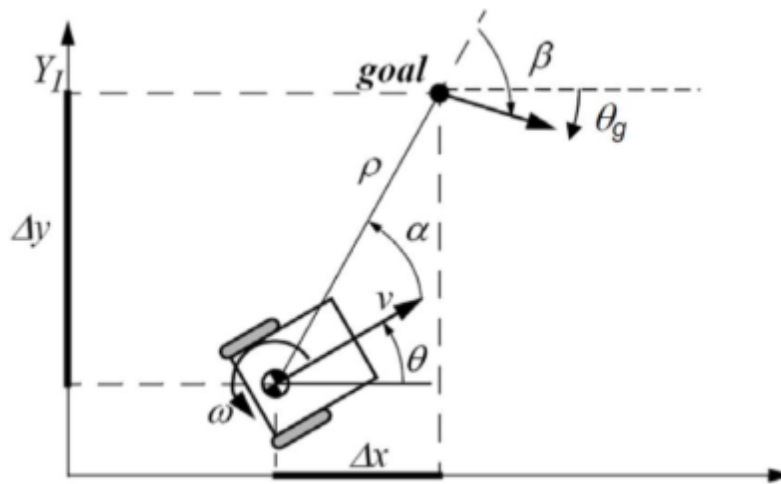


Figure 1: State feedback control onto a reference pose

$$v = k_{\rho} \rho \quad (3)$$

$$\omega = k_{\alpha} \alpha + k_{\beta} \beta \quad (4)$$

Task: Please implement this close-loop position controller within `position_control_node_students.py`. All angles are given in the positive right-hand coordinate frame (counter clockwise).

Validation: Start CoppeliaSim, load scene **Exercise_position_control.ttt** and start the simulation. You should see a circular robot and a set of walls. Now run the launch file `position_control.launch.py`. The robot should start driving towards the target and should come to a stop in the same spot and the same orientation as the target. Select different target positions and orientations within the simulation environment in

order to validate the proper functioning of the controller. For doing so, you can select the *myRobotTarget* in the scene hierarchy and change to the Object/item shift mode by clicking on the corresponding button in the toolbar. After that you can drag-and-drop the target to different positions and orientations. Please note that for strong stability the following conditions must hold (see Chapter 3.6, p.91ff):

$$k_p > 0, \quad (5)$$

$$k_\beta < 0, \quad (6)$$

$$0 < k_a + \frac{5}{3}k_\beta - \frac{2}{\pi}k_p \quad (7)$$

2. Closed-loop Control Enhanced

While the above control law exhibits a simple structure and has a strong stability condition, it also has a few shortcomings. Even though the robot might be able to drive forwards and backwards, the above control law will only output forward velocities and consequently will lead to rather cumbersome motions in some cases. Thus, a method should be derived in order to generalize the above control law for forward and backward velocities.

Looking at the simulation, one can observe that the velocity of the robot is decreasing exponentially when it draws nearer to the target pose. This leads to a very slow motion towards the end of the trajectory. A method should be derived and implemented such that the robot performs the same trajectory as for the above control law but with a constant forward speed. This can be done by observing that the trajectory does not change if the quotient between v and ω is kept constant and thus the output of the control law can be scaled.

Task: Please improve your code within **position_control_node._students.py** to tackle the above shortcomings. The robot should be able to drive in both directions, depending on where it is located with respect to the target pose and it should drive towards it with a constant speed. Implement your changes directly within **position_control_node._students.py**

Validation: Start CoppeliaSim, load scene **Exercise_position_control.ttt** and start the simulation. You should see a circular robot and a set of walls. Now run the script `position_control.launch.py`. The robot should start driving towards the green robot ghost and should come to a stop in the same spot and the same orientation as the target.